PhiS. SimLab: Simulation Challenges
Proyecto: Vibraciones de membrana tipo Chladni

Kevin Mauricio Chacón Ureña - C32060*
Physics in Silico, Escuela de Física, Universidad de Costa Rica
(Dated: 20 de febrero de 2026)

1. Introducción

El experimento clásico de la placa de Chladni es famoso por su capacidad de visualizar el efecto del sonido como una onda en un material sólido. El experimento consiste en usar una placa (generalmente metálica), colocarle arena encima y someterla a vibraciones a una frecuencia específica de una fuente externa (como un parlante o el arco de un violín). Cuando la placa alcanza resonancia con la frecuencia dada, la arena tiende a quedarse donde la placa se mueva menos, mostrando patrones donde se encuentran los nodos de la onda. [2]

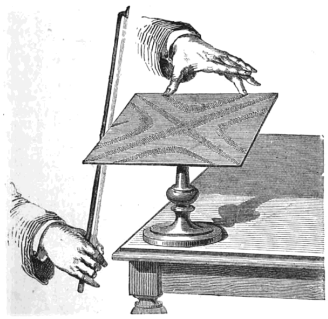

Figura 1: Ilustración del experimento clásico de Chladni

En éste trabajo no se va a simular el experimento clásico de Chladni, pero sí se tomará de inspiración para identificar las líneas nodales en una membrana bidimensional, y ver su comportamiento al someterla a una vibración externa a diferentes frecuencias. Ésto se realizará mediante el lenguaje de programación Python, con las bibliotecas numpy y matplotlib para generar animaciones de la membrana.

2. Método numérico

Para empezar, lo que se busca es resolver la ecuación de onda en 2 dimensiones para una membrana cuadrada, la cuál se ve de la siguiente manera:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

Donde $u(x, y, t)$ es el desplazamiento de la membrana en el punto (x, y) en un cierto tiempo t. Resolver una ecuación así no es tarea fácil, para la elaboración de éste trabajo se usó como referencia un trabajo publicado en la página web: 'Scientific Programming with Python' [1] para la simulación de ondas en dos dimensiones. Resumiremos la justificación del método utilizado en este informe, pero para una explicación más detallada se le invita a visitar la publicación original en las referencias.

Se usó el método común de dividir el espacio en una grilla discreta de puntos y usar diferencias centrales finitas para el cálculo de las derivadas espaciales, de donde llegamos a las siguientes expresiones:

$$\frac{\partial^2 f}{\partial x^2} \approx \frac{f(x+h, y) - 2f(x, y) + f(x-h, y)}{h^2}$$
$$\frac{\partial^2 f}{\partial y^2} \approx \frac{f(x, y+h) - 2f(x, y) + f(x, y-h)}{h^2}.$$

Donde h representa el espaciado entre cada punto de la grilla. Para las derivadas temporales se hace el mismo proceso pero usando δ como variable para las diferencias temporales. A la hora de discretizar la función u como

*Electronic address: kevin.chaconurena@ucr.ac.cr

un arreglo de matrices. Cada matriz representa el espacio donde el índice i corresponde con la coordenada x, y el índice j con la coordenada y, luego se distingue entre las matrices con el índice k que corresponde con la coordenada t.

$$u(t;x,y) = u_{i,j}^{(k)}$$

De ésta manera, dar un paso de h o δ corresponde a sumar 1 en el índice correcto de la siguiente manera:

$$u(t+\delta;x,y) = u_{i,j}^{(k+1)}$$

$$u(t,x+h,y) = u_{i+1,j}^{(k)}, \quad u(t,x,y+h) = u_{i,j+1}^{(k)}.$$

Éstas aproximaciones en la ecuación de onda nos dan que:

$$\frac{1}{\delta^2}\left(u_{i,j}^{(k+1)} - 2u_{i,j}^{(k)} + u_{i,j}^{(k-1)}\right) = \frac{c^2}{h^2}\left(u_{i+1,j}^{(k)} + u_{i-1,j}^{(k)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k)} - 4u_{i,j}^{(k)}\right)$$

Lo cuál se puede acomodar como:

$$u_{i,j}^{(k+1)} = \alpha^2\left(u_{i+1,j}^{(k)} + u_{i-1,j}^{(k)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k)} - 4u_{i,j}^{(k)}\right) + 2u_{i,j}^{(k)} - u_{i,j}^{(k-1)}$$

Donde $\alpha = c\delta/h$. Por supuesto, como con toda ecuación diferencial que trabaje con diferencias espaciales, debemos pensar en lo que ocurre en las fronteras de nuestra membrana. Para el caso de éste proyecto nos interesa que las condiciones sean de tipo Dirichlet, es decir que la frontera de la membrana se mantenga estática en cero, como si estuviera adherida a un soporte.

Usando éste método numérico se hizo un archivo donde se definió una clase para ondas en dos dimensiones que nos permite calcular el estado futuro de la onda conociendo su estado presente (condición inicial).

```
import numpy as np
class Onda2Dim:
    def __init__(self, L, c, h, d): #L: Dimensiones de la membrana
    ↪  [m], c: velocidad de la onda [m/s], h: diferencia
    ↪  espacial, d: diferencia temporal
        self.x0=int(L/h)
        self.y0=int(L/h)
        self.c=c
        self.h=h
        self.d=d
        self.a2= (c*d/h)**2 #valor alpha cuadrado
        self.u = np.zeros((3, self.x0, self.y0)) #función de onda
        ↪  como 3 matrices, en tiempos: k-1, k y k+1
        ↪  respectivamente
        self.cfl=
```

```
    def calcular_tiempo(self):
        u, x0, y0= self.u, self.x0, self.y0
        u[2] = u[1] #Pasamos k -> k-1
        u[1] = u[0] #Pasamos k+1 -> k

        u[0,0,:] = 0 #C. Frontera de Dirichlet
        u[0,-1,:] = 0
        u[0,:,0] = 0
        u[0,:,-1] = 0

        u[0,1:x0-1, 1:y0-1] = self.a2 * (u[1, 1:x0-1,0:y0-2,] +
        ↪  u[1, 1:x0-1, 2:y0] + u[1, 0:x0-2, 1:y0-1] + u[1,
        ↪  2:x0, 1:y0-1] - 4*u[1, 1:x0-1, 1:y0-1]) \
                            + 2 * u[1, 1:x0-1, 1:y0-1] - u[2,
                            ↪  1:x0-1, 1:y0-1] #Calcular u en
                            ↪  k+1
```

Ahora, con el método numérico resuelto, podemos visualizar diferentes casos de la membrana vibrando, tanto en dos dimensiones (mediante un mapa de calor) como en tres dimensiones.

Como nuestro interés principal es el de ver las lineas nodales que se forman, se hizo una paleta de colores personalizada que se puede usar en Matplotlib. Ésta paleta tiene un alto contraste en los valores cercanos al cero, con el propósito de hacer destacar ésta zona.

```
from matplotlib.colors import LinearSegmentedColormap
colors = [
    (1, 1, 1),
    (1, 1, 1),
    (1, 1, 1),
    (1, 1, 1),
    (1, 0, 0.5),
    (0, 0, 0),
    (1, 0.5, 0),
    (1, 1, 1),
    (1, 1, 1),
    (1, 1, 1),
    (1, 1, 1)
]
cmap_nodos = LinearSegmentedColormap.from_list('nodos', colors,
↪  N=512)
```

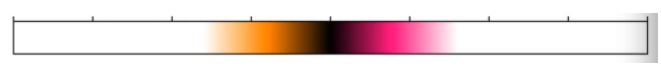

Figura 2: Paleta 'cmap_nodos'

3. Simulación: Perfil Gaussiano

El primer caso que viene a la mente a la hora de pensar en una onda propagándose en una membrana, es el de levantar la membrana en el centro y dejarla ir. El caso más natural de ésto es usar una perturbación con forma de distribución Gaussiana (o normal).

Empezamos importando el archivo donde está la clase que creamos con el método numérico y la paleta de colores. Luego, definimos los parámetros de la simulación. En general se usó una amplitud A de 1, un tamaño de la membrana L de 1 y unas diferencias $h = 0{,}005$ y

$\delta = 0{,}001$, pues son estables para la ecuación de onda en dos dimensiones.

Luego, se crea el objeto de la clase con los parámetros deseados y se decide un tiempo que se quiere que dure la animación.

Luego de ésto, establecemos la condicion inicial de la membrana con el perfil Gaussiano, el cual tiene la forma:

$$u(x,y) = A\exp\left(-\frac{(x-x_c)^2+(y-y_c)^2}{2\sigma^2}\right) \qquad (1)$$

Donde (x_c, y_c) es el centro y σ es el grosor de la gota gaussiana.

En código, todo lo mencionado se ve así:

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from Onda2Dim import Onda2Dim
from Onda2Dim import cmap_nodos
from matplotlib.animation import FFMpegWriter

A = 1 #Amplitud
sigma = 11 #Ancho perfil Gaussiano (en unidades de grilla)

L = 1
h=0.005
d = 0.001

sim = Onda2Dim(L, 2, h, d)
t_fin=1.5

#Perfil Gaussiano inicial
xc, yc=sim.x0//2, sim.y0//2 #centro de la gota
for i in range(sim.x0):
    for j in range(sim.y0):
        sim.u[0,i,j] =
        ↪  A*np.exp(-((i-xc)**2+(j-yc)**2)/(2*sigma**2))
        sim.u[1,i,j] = sim.u[0,i,j]
```

Se asigna la forma de la gota para $u^{(0)}_{i,j}$ y $u^{(1)}_{i,j}$ para establecer que parte del reposo. Luego, se procede haciendo la animación con el mapa de color como la hecha en la referencia [1].

```
def actualizar_anim(i):
    sim.calcular_tiempo()

def init():
    return img,

fig, ax = plt.subplots(figsize=(8, 8),
    dpi=200)
img = ax.imshow(sim.u[0], vmin=-A, vmax=A, cmap=cmap_nodos)

contador = plt.text(
    0.0, 0.0, '',            # posición en coordenadas del eje
    fontsize=12,
    verticalalignment='top', color='white')

ax.set_xticks([])
ax.set_yticks([])
plt.xlabel('x [m]')
plt.ylabel('y [m]')

cbar = plt.colorbar(img, ax=ax)
cbar.set_label("Amplitud de la onda [m]")

ax.set_title('Membrana estilo Chladni: Perfil Gaussiano en el
↪  centro')

def animate(i):
    actualizar_anim(i) #i: frame actual
    img.set_data(sim.u[0])
    contador.set_text(f"t = {i*d:.2f} s")
    return img, contador

interval, nframes = sim.d/10, int(t_fin/d)
anim = animation.FuncAnimation(fig, animate, frames=nframes,
                               repeat=False,
                               init_func=init, interval=interval,
                               ↪  blit=True)

#plt.show()

writer = FFMpegWriter(
    fps=int(0.1/d),   # frames por segundo coherente con tu dt
    metadata=dict(artist='Matplotlib'),
    bitrate=1800
)

anim.save("Onda2Dimension (Gaussiana).mp4", writer=writer)


import winsound
winsound.Beep(1500, 700)
```

El código de ésta manera genera un archivo mp4 con la animación (y hace un sonido al terminar de ejecutarse). También se agregó a la animación un contador que dice el tiempo actual. Si se desea que solo se muestre la animación sin guardarse, es solo cuestión de quitar el numeral antes de `plt.show()` y comentar con `'''` todo lo que va luego de eso.

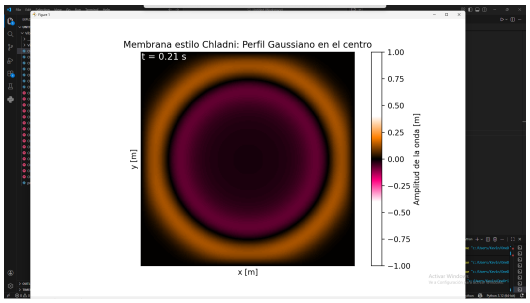

Figura 3: Captura de pantalla de la simulación del Perfil Gaussiano en dos dimensiones

El código para la animación en tres dimensiones se hizo con ayuda de un video de youtube sobre mapeo en 3D [3], y se ve de la siguiente manera:

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from Onda2Dim import Onda2Dim
from Onda2Dim import cmap_nodos
from matplotlib.animation import FFMpegWriter

A = 1 #Amplitud
sigma = 11 #Ancho perfil Gaussiano

L = 1
h=0.005
d = 0.001
```

```
sim = Onda2Dim(L, 2, h, d)
t_fin=1.5

#Perfil Gaussiano inicial
xc, yc=sim.x0//2, sim.y0//2 #centro de la gota
for i in range(sim.x0):
    for j in range(sim.y0):
        sim.u[0,i,j] =
        ↪  A*np.exp(-((i-xc)**2+(j-yc)**2)/(2*sigma**2))
        sim.u[1,i,j] = sim.u[0,i,j]

# Configuración de la figura 3D
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# Crear la malla de coordenadas
x = np.linspace(0, L, sim.x0)
y = np.linspace(0, L, sim.y0)
X, Y = np.meshgrid(x, y)

# Configurar la superficie inicial
Z = sim.u[0]  # Amplitud inicial
surf = ax.plot_surface(X, Y, Z, cmap=cmap_nodos,
                       vmin=-A, vmax=A,
                       linewidth=0, antialiased=True)

ax.set_xlabel('x [m]')
ax.set_ylabel('y [m]')
ax.set_zlabel('Amplitud [m]')
ax.set_title('Membrana estilo Chladni: perfil Gaussiano en el
↪  centro')
ax.set_zlim(-A, A)
ax.view_init(elev=30, azim=45)

cbar = plt.colorbar(surf, ax=ax, shrink=0.5, aspect=10)
cbar.set_label("Amplitud de la onda [m]")

contador = ax.text2D(0.02, 0.95, '', transform=ax.transAxes,
                     fontsize=12, color='black')


ax.set_title('Membrana estilo Chladni: Perfil Gaussiano en el
↪  centro')

def actualizar_anim(i):
    sim.calcular_tiempo()

def animate(i):
    global surf

    actualizar_anim(i)
    Z_new = sim.u[0]

    surf.remove()
    surf = ax.plot_surface(
        X, Y, Z_new,
        cmap=cmap_nodos,
        vmin=-A, vmax=A,
        linewidth=0,
        antialiased=True
    )

    contador.set_text(f"t = {i*d:.3f} s")
    return surf, contador

interval, nframes = 20, int(t_fin/d)

anim = animation.FuncAnimation(fig, animate, frames=nframes,
                               repeat=False,
                               interval=interval, blit=False)

#plt.show()

writer = FFMpegWriter(
    fps=int(0.1/d),   # frames por segundo coherente con tu dt
    metadata=dict(artist='Matplotlib'),
    bitrate=1800
)
anim.save("Onda3Dimension (Gaussiana).mp4", writer=writer)

import winsound
winsound.Beep(450, 700)
```

Éste código funciona igual que el anterior, con la diferencia de generar animaciones en tres dimensiones.

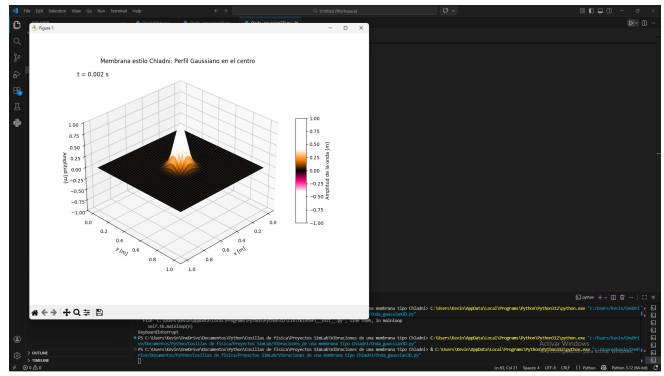

Figura 4: Captura de pantalla de la simulación Perfil Gaussiano en tres dimensiones

4. Simulación: Modos de vibración

En el caso unidimensional de una cuerda vibrando, sabemos que hay diferentes modos de vibración que permiten que la cuerda presente una onda estacionaria con sus frecuencias concretas. En el caso bidimensional de la membrana tenemos el mismo fenómeno, pero ahora con dos números que caracterizan el modo de vibración en vez de uno.

La forma de éstos modos está dada por la siguiente expresión:

$$u(x,y) = A\sin\left(\frac{m\pi}{L}x\right)\sin\left(\frac{n\pi}{L}y\right) \tag{2}$$

Donde (m,n) corresponde a los números de modo, siendo la cantidad de antinodos a lo largo del eje x y y respectivamente.

Se pueden sumar dos modos de vibración distintos para obtener diferentes ondas estacionarias. Para representar ésta posibilidad se incluyeron las variables (m_1, n_1) que si son ambas diferentes a cero suman un nuevo modo de vibración, tal que:

$$u(x,y) = A\left(\sin\left(\frac{m\pi}{L}x\right)\sin\left(\frac{n\pi}{L}y\right) + \sin\left(\frac{m_1\pi}{L}x\right)\sin\left(\frac{n_1\pi}{L}y\right)\right) \tag{3}$$

[4]

El código es muy parecido al de el perfil Gaussiano, con la diferencia de la expresión que se usa para establecer las condiciones iniciales, lo cuál se ve así:

```
m, n= 3,2 #modos de vibración

m1,n1=0,0

for i in range(sim.x0): #posición inicial del modo
    for j in range(sim.y0):
        sim.u[0,j,i] = A * (np.sin(m*np.pi*i*h/L) *
        ↪  np.sin(n*np.pi*j*h/L)+np.sin(m1*np.pi*i*h/L) *
        ↪  np.sin(n1*np.pi*j*h/L))
        sim.u[1,j,i] = sim.u[0,j,i].copy()
```

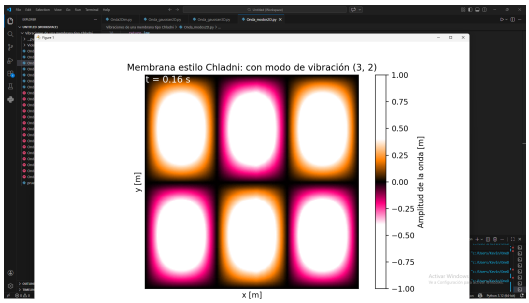

Figura 5: Captura de pantalla de la simulación en el modo de vibración (3,2) en dos dimensiones

Figura 6: Captura de pantalla de la simulación en el modo de vibración (3,1)+(1,3) en dos dimensiones

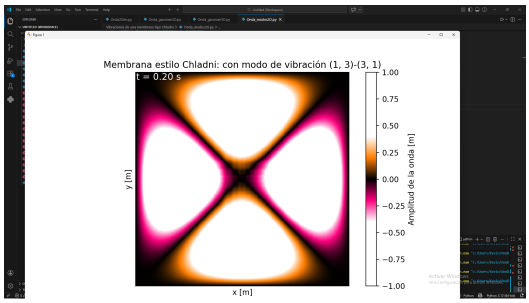

Figura 7: Captura de pantalla de la simulación en el modo de vibración (1,3)-(3,1)) en dos dimensiones

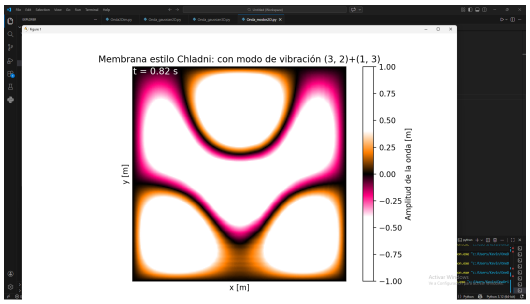

Figura 8: Captura de pantalla de la simulación en el modo de vibración (3,2)+(1,3) en dos dimensiones

Para la animación 3D, de nuevo, el código es muy parecido al de el perfil Gaussiano.

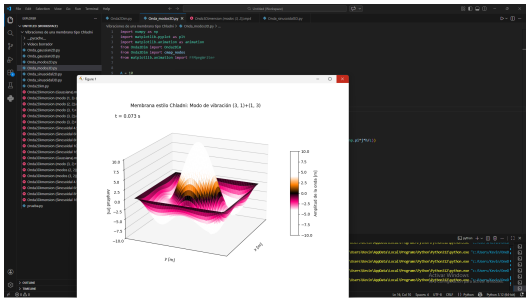

Figura 9: Captura de pantalla de la simulación en el modo de vibración (3,1)+(1,3) en tres dimensiones

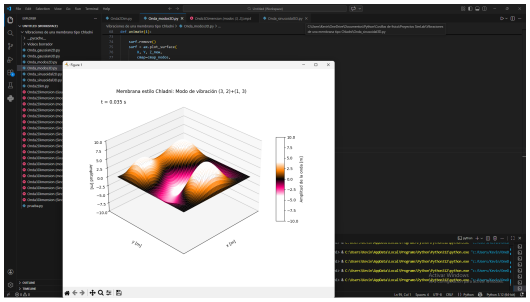

Figura 10: Captura de pantalla de la simulación en el modo de vibración (3,2)+(1,3) en tres dimensiones

5. Simulación: Señal sinusoidal forzada

El último caso es el más similar al experimento que se describió en la introducción. Consiste en imaginar que agarramos la membrana en su centro y la forzamos a moverse en MAS, causando ondas forzadas. La señal forzada sería análoga ala fuente externa del experimento de

Chladni, y esperaríamos observar diferentes líneas nodales dependiendo de su frecuencia.

Para aplicar la señal forzada, lo que se hizo fué modificar la función `actualizar_anim(i)`, para incluir la condición de que el centro de la membrana debe moverse según:

$$u(t, L/2, L/2) = A\sin(\omega t) \tag{4}$$

Donde ω se obtiene a partir de la frecuencia f que le suministremos.

```
def actualizar_anim(i):
    sim.u[0, sim.x0//2, sim.y0//2] = A * np.sin(i*d* w) #una señal
    ↪  sinosoidal en el centro
    sim.calcular_tiempo()
```

El resto del código es bastante similar a los de las otras dos simulaciones.

Lo más interesante que resultó de ésta simulación fue el esperar a que se estabilizara la señal y observar como se formaban patrones de lineas nodales similares a los del experimento de Chladni.

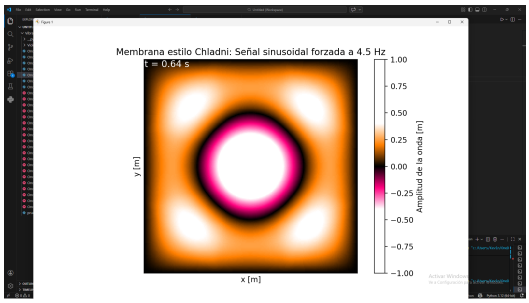

Figura 11: Captura de pantalla de la simulación con señas sinusoidal forzada a 4.5Hz en dos dimensiones

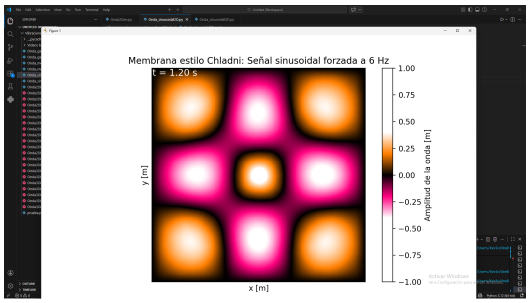

Figura 12: Captura de pantalla de la simulación con señas sinusoidal forzada a 6Hz en dos dimensiones

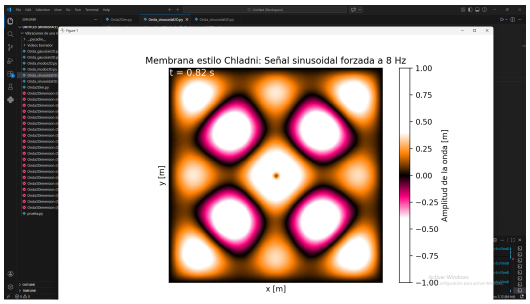

Figura 13: Captura de pantalla de la simulación con señas sinusoidal forzada a 8Hz en dos dimensiones

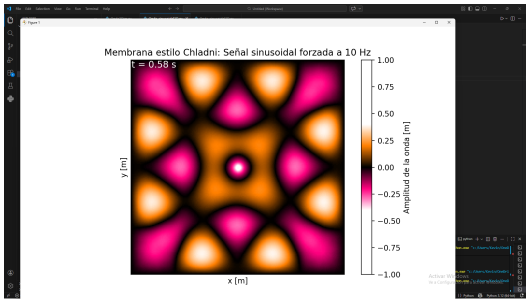

Figura 14: Captura de pantalla de la simulación con señas sinusoidal forzada a 10Hz en dos dimensiones

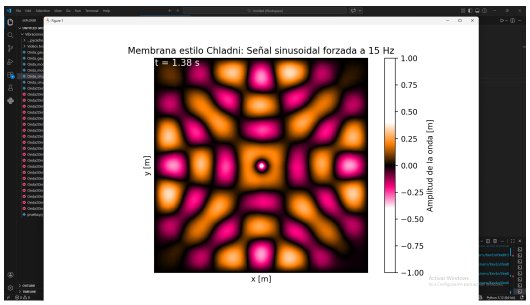

Figura 15: Captura de pantalla de la simulación con señas sinusoidal forzada a 15Hz en dos dimensiones

Luego, se hizo lo mismo pero en tres dimensiones.

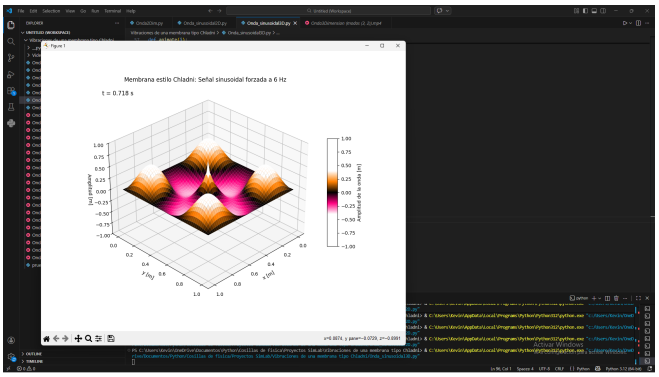

Figura 16: Captura de pantalla de la simulación con señas sinusoidal forzada a 6Hz en tres dimensiones

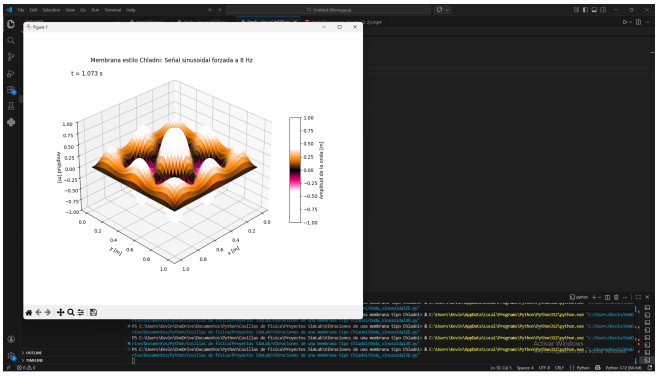

Figura 17: Captura de pantalla de la simulación con señas sinusoidal forzada a 8Hz en tres dimensiones

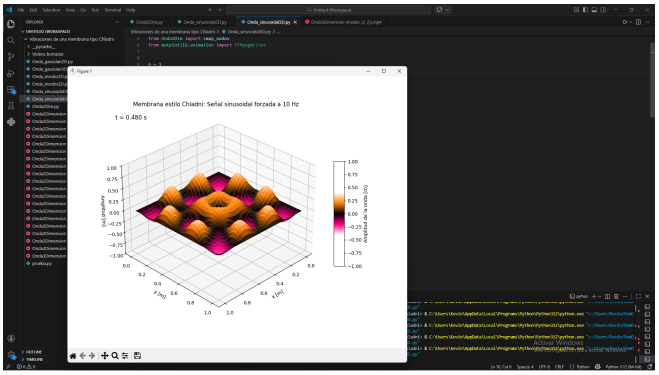

Figura 18: Captura de pantalla de la simulación con señas sinusoidal forzada a 10Hz en tres dimensiones

6. Conclusiones

El proyecto fue una gran oportunidad para familiarizarse con la confección de animaciones divulgativas para la ciencia, y un puente para aprender de temas de acústica y ondas en general.

El proyecto tiene varias oportunidades de mejora, principalmente simulando una membrana circular ahora en vez de cuadrada, y haciéndolo más accesible al público, quizás con una interfaz de usuario o haciéndolo correr en línea para que más personas puedas acceder a él.

Referencias

[1] The two-dimensional wave equation, 2 2024. URL `https://scipython.com/blog/the-two-dimensional-wave-equation/`.

[2] J. C. P. Campuzano. Chaldni patterns interactive simulations, 6 2022. URL `https://www.dynamicmath.xyz/chladni-patterns/`.

[3] P. Central. Advanced 3d plotting techniques in matplotlib, 1 2025. URL `https://youtu.be/L9-zdJ1pkJc?si=Oc7V_kN4tsHhggfg`.

[4] G. P. i. A. Daniel A. Russell, The Pennsylvania State University. Vibrational modeshapes of a rectangular membrane (fixed at the edges), 10 2018. URL `https://www.acs.psu.edu/drussell/Demos/rect-membrane/rect-mem.html`.